Overbooked

Rental Management SaaS - Technical Documentation

Full-stack, multi-tenant rental software (Next.js 16 + Supabase/Postgres). Built from the TRENDZ admin panel (forked 22 Oct 2026) into a sellable product for many rental shops. Scanned from the repository on 12 Nov 2026.

Item	Value
Repository	zypher-solutions/Overbooked (GitHub), branch main, 115 commits, PRs up to #25
Code size	about 29,400 lines TypeScript/TSX (src) + 9,250 lines SQL migrations (42 files) + 2,450 lines SQL tests
Database	about 35 tables, about 194 plpgsql functions, 67 RLS policies, 22 pgTAP files (575 planned assertions)
Browser tests	5 Playwright specs (30 tests)
First target market	Bridal and groom (clothing) rental shops in India (INR, GST)
Later modules	vehicle (cars/bikes), stay (resorts), then a mobile app

How to use this PDF: Part 1 is the 2-minute pitch. Parts 2-5 are the architecture and stack. Parts 6-9 are files, frontend and backend. Part 10 is features. Part 12 is likely interview questions with short answers.

1. The 2-minute pitch

- **What:** SaaS that rental shops use to run bookings, hand-over, returns, deposits, payments, stock, customers and finance. One shop never sees another shop's data.
- **Who:** built by Zypher Solutions. Zypher sets each shop up and bills it from a separate console. Shops are the customers.
- **Origin:** grew out of TRENDZ (an admin panel for a bridal/groom rental shop). Forked, then re-architected for many shops.
- **Key design decision:** no separate backend server. All business rules (pricing, availability, money invariants, permissions) live in Postgres functions with Row Level Security. The web app, a future mobile app and anything else get identical rules.
- **Multi-tenancy:** one shared database; every business row carries `shop_id`; a restrictive RLS policy pins each user to `current_shop_id()`.
- **Extensibility:** business types (bridal, groom, resort, car, bike) are data (templates). Screens for a business type live in `src/modules/<name>`; a registry picks the module per shop; ESLint stops modules importing each other.
- **Quality:** every DB change is a migration; pgTAP database tests; Playwright browser tests; GitHub Actions CI; encrypted daily backup workflow; strict CSP/security headers; two-step (MFA) sign-in for Zypher staff; tamper-proof activity log written by the database.

2. Tech stack

Layer	Technology (version from package.json)	Used for
Framework	Next.js ^16.2 (App Router, Turbopack), React ^19.3, TypeScript ^5.8	Pages, routing, server actions, route guards
Styling / UI	Tailwind CSS 4, shadcn/ui on Radix UI primitives, class-variance-authority, tailwind-merge, tw-animate-css, lucide-react icons, sonner (toasts), vaul (drawers), cmdk, embla carousel	Design system, 40+ UI components in src/components/ui
Forms / validation	react-hook-form + @hookform/resolvers, own validate.ts (email, Indian phone, names) shared by browser and server actions	Forms with the same checks client + server
Dates / charts	date-fns 4, react-day-picker 9, recharts 2	Calendar, date pickers, dashboard charts
Backend / DB	Supabase: Postgres, Auth, RLS, RPC (plpgsql), Storage, Realtime. @supabase/supabase-js ^2.112, @supabase/ssr	All data, auth, rules, files
Postgres extras	btree_gist (exclusion constraint), pg_cron (nightly job), pgTAP (tests)	No double-booking; auto-pause overdue shops
PDF / files	@react-pdf/renderer (ledger PDF), xlsx (SheetJS) + papaparse (Excel/CSV import-export), qrcode-generator, html5-qrcode (camera scan), browser-image-compression	Ledger, bulk sheets, QR tags, scanning, photo upload
Testing	Playwright ^1.64 (browser), pgTAP via 'supabase test db' (database), scripts/check-bulk.ts	Quality gates
Tooling	ESLint 9 + typescript-eslint, Prettier 3, Dependabot, GitHub Actions (ci.yml, backup.yml)	Lint, format, CI, backups
Platform	Windows 11 dev, Node 22 in CI, Docker only for local Supabase throwaway checks	Environment

Notes to remember: there is no Python in the project (this PDF was generated outside it). The only outside host the app talks to is its own Supabase project (set in the CSP). The cloud Supabase project is shared by three developers.

3. Architecture

```
Browser (Next.js: shop app + Zypher console)
   |  reads tables directly (RLS-filtered), writes ONLY via database functions (RPC)
   v
Supabase
  - Auth     : email+password, invite links (server actions use the service key), MFA (aal2)
  - Postgres : tables + RLS + rental engine (plpgsql) + triggers + pg_cron
  - Storage  : product photos (public), handover photos (private), one folder per shop
  - Realtime : rentals, payments, units, reservations
```

Request flow

- **src/proxy.ts** (Next.js proxy/middleware): builds a Supabase server client from cookies, calls `getClaims()` (verifies JWT signature locally, refreshes expired tokens, avoids a network call per page). No session means redirect to /login; session on /login means redirect to /. /welcome (invite landing) is public.
- **Client reads:** `createClient()` (src/lib/supabase/client.ts) then `.from(table).select()`, filtered by RLS.
- **Client writes:** `call(fn, args)` in src/lib/shop/api.ts wraps `supabase.rpc()` and turns DB errors into plain-language Errors.
- **Server actions** (src/app/actions/staff.ts, zypher.ts): only for things needing the service-role key (create auth users, invite links, reset emails). They re-check the caller via `get_my_role()` and call a DB check function first; the DB re-checks again.
- **Global state:** AuthContext (session, profile, shop) and ShopContext (branches, categories, templates, add-ons, attributes, feature/setting helpers, businessModule).

Security layers (defence in depth)

- **RLS** on every table; restrictive policy keeps users inside `current_shop_id()`. Zypher admins have NULL shop id, so no business table is open to them.
- **Composite foreign keys** (shop_id, x_id) make it impossible to link rows of two shops.
- **app_rpc role** (NOLOGIN, NOBYPASSRLS): shop functions run as it, so even a bug inside a function cannot cross shops. Zypher console functions run as owner and check `is_platform_admin()`.
- **Least privilege** (migration 018) and **two-step sign-in** (migration 017: `_mfa_ok()` requires aal2 for Zypher staff/support).
- **Support access consent** (032/036): Zypher support login reaches a shop only while the owner has switched support access on; every switch audited.
- **HTTP headers** (next.config.mjs): CSP (self + Supabase only, no framing, no plugins), HSTS, nosniff, X-Frame-Options DENY, Referrer-Policy, Permissions-Policy (camera self only), COOP.
- **Validation twice:** same validate.ts in browser and server actions; DB CHECK constraints are the final word.

4. Roles and permissions

Role	Who	Can do
super_admin	Zypher staff	Console only: create shops, plan/fee, pause, business types, setup (branches/staff) until owner signs in. NEVER sees a shop's bookings, customers, money, products or photos.
support	Zypher login for ONE shop	No access until owner enables 'Zypher support access'. Then works like owner except no money corrections/deletions. Does not use the shop's login limit. MFA required.
owner	Shop owner	Everything in the shop: all branches, money, corrections (with reason), settings, staff and branches, activity log.
manager	Branch manager	Own branch: bookings, catalog, units, branch money reports, expenses, block/unblock customers.
staff	Counter staff	Own branch: bookings, hand over, return, customers. No money reports.

Owner-tunable **Staff controls** (migration 039, enforced in DB): discount limits for staff/manager, who may void a payment, lower a suggested late fee, cancel a paid booking, give deposit back early, and how far back a return may be dated (2/6/24 h).

5. Business-type modules

- **Templates are data** (`business_templates`): kind (wearable by day, vehicle by hour/day + km, room by night + guests), labels, fields, checks, rate types, turnaround, default features. Five seeded: bridal, groom, resort, car, bike (later consolidated into one clothing template, migration 015).
- **Module registry** (`src/modules/registry.ts`): `ShopModule` = nav, Home screen, tagline, own screens, replace() map for shared screens, ProductForm, importSkus, BookingForm, bookingByPiece, PrintBill. Two entries: `clothing` and `standard` (cars/bikes/resorts, older generic screens).
- **Module key** chosen in ShopContext (`businessModule`): shops whose business types are all clothes use clothing.
- **Rules:** a module never imports another (enforced by ESLint); business rules stay in the DB; client-specific requests become template setting, then per-shop feature switch (`shops.settings.features`), then module code behind a default-off switch. Never a branch or copy for one client.

- **Route files are one-liners:** `export { default } from '@/modules/clothing/screens/X'`, so URLs stay flat while code lives in the module.

6. File and folder map

Root

File	Purpose
CLAUDE.md / AGENTS.md	Project rules for AI assistants: hard rules (never touch TRENDZ, never reset shared DB, migrations for every change), activity-log checklist, commands
package.json	Scripts: dev, build, start, lint, typecheck, check:bulk, test:fixtures, test:reset, test:db, test:e2e, format, types (generate DB types)
next.config.mjs	CSP + security headers, Supabase image patterns, serverExternalPackages canvas (for react-pdf)
eslint.config.js	Includes the rule that blocks cross-module imports (MODULES list)
playwright.config.ts	Loads .env.local/.env.test, 1 worker, starts npm run dev
components.json / postcss.config.mjs	shadcn config; Tailwind 4 PostCSS plugin
.github/	workflows/ci.yml (app + database + browser jobs), backup.yml (encrypted daily pg dump), dependabot.yml, CODEOWNERS, PR template
scripts/	check-bulk.ts (tests bulk sheet rules), load-data.sql + load-test.mjs (load/performance test on Docker)

src/app (routes)

Path	What it is
layout.tsx, error.tsx, global-error.tsx, not-found.tsx, manifest.ts, icon.svg	Root layout, error boundaries, PWA manifest
login/, mfa/, welcome/	Sign-in, 6-digit code (TOTP) step, invite/reset landing (sets password then MFA)
(shop)/	The shop app. layout.tsx wraps ShopShell. Routes: / (dashboard), book, bookings, active-rentals, history, all-rentals, rentals/[id], calendar, appointments, customers(/[id]), catalog (/new, /[id], /bulk, /import, /earnings), cleaning, scan, checkout, finance, branches, staff, settings, account, admin/(activity, attributes, sku)
zypher/	Zypher console (server-guarded layout): overview, shops, shops/new (4-step wizard), shops/[id], billing, templates, audit, team, settings
print/	Printable pages: rental/[id] (receipt or GST tax invoice), document/[id] (credit note/supplementary bill), tags (QR stickers), zypher-receipt/[id]
actions/	Server actions: staff.ts (inviteStaff, newInviteLink...), zypher.ts (create shop, support access...)

src/lib

File	Purpose
lib/supabase/client.ts, server.ts, admin.ts	Browser client, cookie-based server client, service-role client (server only)
lib/overbooked/AuthContext.tsx	Session, profile (role, branch, shop), Shop object (plan, status, settings...), sign-out, session-ended dialog
lib/overbooked/brand.ts	Sets shop brand (name/logo) in UI
lib/shop/ShopContext.tsx	Loads branches, categories, templates, add-ons, attributes once; exposes feature(), setting(), branchSetting(), word(), templateOf(), businessModule, refresh()
lib/shop/api.ts	call() RPC wrapper, RENTAL_LIST_COLUMNS, loadRental() (one rental with lines, add-ons, charges, checks, reservations, payments, documents)
lib/shop/types.ts, format.ts	TypeScript types; formatting in the shop's time zone, rupees(), errorText()
lib/shop/nav.ts	Shared nav types, ModuleKey, standardNav
lib/shop/useCatalog.ts, searchRentals.ts, whatsapp.ts, activity.ts	Catalog loader; rental search; WhatsApp message builder (wa.me links, 4 message kinds); logCartEvent for browser-only cart actions

File	Purpose
lib/server/invites.ts	getCaller() (role from DB, never trusts browser), siteOrigin(), createInviteLink()
lib/validate.ts, utils.ts, database.types.ts	Form checks; cn(); generated Supabase types

src/components

Folder	Contents
ui/	shadcn/Radix primitives (accordion, dialog, table, sidebar, calendar, chart, drawer, tabs...)
overbooked/	CameraScanner, ConfirmDialog, OfflineBanner, PasswordField, primitives
shop/	ShopShell (frame/menu/route guard), Today, useBookingForm (shared booking state + rules), HandoverDialogs, RentalDialogs, EditRentalDialog, BookingPaymentSummary, BillsPanel (invoices/credit notes), ProductEditor, UnitsPanel, MeasurementsPanel, RemindersPanel, CustomerPicker/Form, Appointments, ActivityLog, Subscription, QrCode, DateInput, RentalList
zypher/	ZypherShell (console frame), kit (shared console widgets)

src/modules/clothing

Item	Purpose
nav.ts	Menu: Work (Dashboard, Finance, Bookings, Active rentals, History, Calendar, Appointments) / Inventory (Inventory search, Cleaning & repairs, Products, Add New, Bulk add & edit, Categories, Accessories, Earnings by product) / People / Admin (Staff, Attributes, SKU, Branch management, Settings, Activity log)
screens/	Bookings, ActiveRentals, History, AllRentals, InventorySearch, BookingForm, Finance (1,100 lines), BulkProducts, ServiceJobs, ProductEarnings, Attributes, AttributeValues, SkuFormat
components/	TrendzDashboard, ProductWizard (3 steps), ProductTable, ProductDetail, StockCard, RentProductDialog, CheckoutCart, CheckoutPayment, CartSwitcher, ExpensesPanel
cart.tsx, cart-storage.ts	Per-person-per-shop cart kept on the device (localStorage); multi-cart
sku.ts, bulk.ts, sheet.ts	SKU format (BR-001, BR-001-GOLD-M) and CSV numbering; bulk add/edit rules; Excel/CSV via SheetJS
ledgerPdf.tsx, print/ClothingBill.tsx	Finance ledger PDF; clothing receipt / tax invoice layout

7. Frontend in detail

Shell, guard and navigation

- **ShopShell.tsx** picks the module from the registry, renders the side menu from `nav(ctx)`, and sends someone who opens a hidden page by URL back to Today (the menu hides, the shell also guards).
- Mobile: `use-mobile` hook, sidebar/drawer; PWA manifest; OfflineBanner shows loss of connection.
- **Zypher console** layout is server-guarded (role checked on the server, not just hidden links).

Key screens (clothing module)

Screen	Behaviour worth knowing
Dashboard (TrendzDashboard)	Pickups today / Returns due today / Overdue cards pick the list below; Hand over, Return, WhatsApp buttons; To collect; sortable list. Uses Realtime.
Inventory Search (/scan)	Scan tag by camera (html5-qrcode) or USB scanner, or type name/SKU/piece code/booking no.; stock per branch (Total/Avail/Rented); Add to Cart; scanning a returning piece opens its return.
Booking form + cart	Dates first, each piece chosen, cleaning days per booking, no pickup in the past, customer + payment locked until every item is free. State/rules in useBookingForm.ts (576 lines) shared; clothing view in BookingForm.tsx.
Rental page (/rentals/[id])	Edit before pickup, hand over with per-piece checks and photos, return with suggested charges, money box, extend, swap piece, cancel/no-show, owner corrections with reason, WhatsApp messages, invoices/credit notes.
Return (ReturnPieces)	Each piece scanned, typed or ticked; a missing piece must be marked lost.

Screen	Behaviour worth knowing
Finance	Earned, collected, outstanding, deposits held vs previous period; expenses and profit; daily close by payment method; by branch; ledger CSV and PDF. Loads rentals in range in the browser (to move to a DB function for big shops).
Products	3-step ProductWizard (basic info, attributes, variations and pieces per branch; pieces get QR codes), All Products table, ProductDetail, Bulk add/edit spreadsheet with Excel paste and import/export, Categories, Accessories (add-ons), Earnings by product, CSV import.
Cleaning & repairs	Service job board per piece: to do, away, back, done, cancelled; cost becomes an expense; time-boxed hold blocks booking.
Customers / Appointments / Calendar	Customer history, owed amount, ID, block; measurements; fittings/trials; timeline per piece.
Admin	Staff management, Branch management with Settings tabs (Details, Receipts & tax, Late returns, Staff controls), Attributes, SKU format, Activity log.

Frontend patterns

- Client components using hooks + Supabase client; server components only for guards and layouts.
- **Plain-language errors:** DB functions raise messages written for shop owners; `errorText()` shows them as-is.
- **Time zones:** all formatting through shop timezone (`format.ts`), default set in DEFAULT_TZ.
- **Per-branch settings:** `branchSetting(branchId, key, fallback)` falls back branch to shop to default.
- **Printing:** browser print pages under /print; PDF only for the ledger (react-pdf).

8. Backend (Supabase / Postgres) in detail

Main tables (about 35)

Group	Tables
Platform	shops, branches, profiles, staff_invites, business_templates, platform_settings, shop_counters, shop_payments (Zypher billing), audit_log, support_access, support_visits
Catalog	categories, products, product_variations, variation_rates, units (physical pieces), set_items, add_ons, shop_attributes, shop_attribute_values
Rentals	customers, customer_measurements, rentals, rental_lines, unit_reservations, rental_add_ons, rental_charges, rental_checks, payments, rental_documents, rental_reminders, appointments
Other	expenses, price_changes, service_jobs, activity_log

Rental lifecycle (engine functions)

```
create_rental  -> status reserved   (quote recalculated in DB; units reserved for period + turnaroun
d)
start_rental   -> active            (per-unit checks, licence for vehicles, money taken; refuses a h
eld booking)
complete_rental-> completed         (checks, charges from return_suggestions, deposit kept/returned,
 unit freed after turnaround, damaged to service)
cancel_rental  -> cancelled / no_show (fee, refund, reason)
later: add_rental_charge, record_rental_payment, void_payment, admin_* corrections (owner + reason)
```

- **Booking number:** PREFIX-YYYY-NNNN per shop (counter table). **Invoice:** PREFIX/FY/NNNN (max 16 chars) fixed on issue. **Zypher receipt:** ZYP-FY-NNNN.
- **Money:** numeric(14,2), never floats. `payments` typed (payment, refund, deposit, deposit_refund, deposit_applied); triggers keep rental money columns; total is computed.
- **No double booking:** exclusion constraint (btree_gist) on `unit_reservations` time ranges (rental time + turnaround).
- **Rates:** variation_rates with hour, day, night, extra_km, extra_hour, extra_person, driver_day, late_penalty. `is_override` marks negotiated price.

Invariants enforced by the database

- total = lines + add-ons + charges - discount (cancelled rental's total = its fee)
- 0 <= paid <= total, so balance due is never negative
- deposit refunded + kept <= deposit received; no negative amounts; end after start; no overlapping use of a unit

- Finance/dashboard sum the payment records themselves, so they always match; voided payments are excluded everywhere

Function families (about 194 total)

Area	Examples
Booking	quote_rental, branch_availability, find_available_units, available_units, create_rental, update_rental, extend_rental, cancel_rental, swap_unit, update_rental_notes, mark_reminder_sent
Pickup / return	start_rental, return_suggestions, complete_rental, set_handover_hold, late_clashes
Money	record_rental_payment, add_rental_charge, void_payment, admin_set_discount, admin_update_payment, admin_delete_payment/charge/rental
Documents	issue_invoice, issue_credit_note, supplementary bill trigger _invoice_followups
Catalog	save_product, save_product_with_pieces, save_set_items, import_catalog, delete_or_deactivate_product, set_unit_status, set_product_late_fee
Service	create_service_job, update_service_job, _job_holds (time-boxed hold in availability)
Reports	inventory_status, dashboard_summary, open-money list for Finance
Setup / staff	setup_branches, setup_save_branch, setup_people, setup_set_person, setup_check_invite, setup_check_person, update_my_shop, update_branch_settings, set_support_access
Zypher console	create_shop, update_shop, record_shop_payment, platform_overview, console_summary, shop_detail, pause_overdue_shops (pg_cron nightly 19:30 UTC)
Auth helpers	get_my_role, current_shop_id, is_platform_admin, is_support, _mfa_ok, mfa_required, _require_role, _require_correction, _require_rule_role
Logging	_act_log, _act_change, _act_* per-table functions, log_account_event, log_setup_event, log_cart_event, activity_people

Activity log (design worth explaining)

- Table `activity_log` written ONLY by the database through triggers (`trg_activity` + `_act_<table>()`), so it cannot be skipped or forged from the browser.
- Each entry: shop, branch, area, action, level (0 created, 1 changed, 2 named event, 3 deleted), record type/id, label, one plain sentence, important flag, before/after changes, snapshot for deletions.
- One save is one entry (child rows logged under the parent). Reads, page views, searches are not logged. `_act_log` swallows its own errors so logging never blocks work.
- Owner-only screen at /admin/activity with filters and spreadsheet download. Browser-only actions (cart) go through `log_cart_event`.

Storage and Realtime

- Buckets: product photos (public), handover photos (private); one folder per shop; size limits (migration 019).
- Realtime on rentals, payments, units, reservations (migration 003).

9. Migration timeline (42 files, 20261101000000 to 041)

#	What it added
000-003	Platform (shops, branches, profiles, billing), catalog, rentals and money triggers, RLS + storage + realtime
004-007	Templates, pricing/availability engine, workflow (book, pickup, return), admin/corrections/reports
008-010	Console reads, app helpers, operations (change booking, sets, reminders, receipts, auto-pause, measurements)
011-016	Zypher-only setup, cleaning days per booking, SKU format for clothes, reactivate product, single clothing template, setup-only console
017-021	Two-step sign-in, least privilege, storage limits, private shop notes, setup in console
022-027	Launch rules, expenses, save product with pieces, invoice numbers, trial in console, finance open money
028-031	Archive variations, inventory tools (bulk), shop attributes + grants
032-036	Support access consent, owner runs staff, activity log (+gaps), support visits
037-041	Late-return rules, per-branch settings, staff controls + void payment, invoices/credit notes/receipts, cleaning and repairs (service_jobs)

10. Feature list

Shop app

- Dashboard with daily work queues; Finance with expenses/profit/daily close/ledger PDF-CSV
- Booking with live availability, negotiated price, advance and deposit; multi-cart; WhatsApp confirmations and reminders (wa.me links)
- Hand over and return with per-piece checks, photos, scan, lost-piece handling, late fee suggestions with workings
- Payments, refunds, deposits, voiding a wrong payment, owner corrections with reason
- Receipts with numbers; GST tax invoices by button (at booking or after return), credit notes, supplementary bills
- Products: wizard, variations with rates, pieces with QR codes, bulk spreadsheet add/edit, Excel/CSV import and export, SKU format, attributes, earnings by product
- Cleaning and repairs jobs with time-boxed holds; auto cleaning job after return
- Customers (history, owed, ID, block), measurements, appointments, calendar timeline
- Staff management, branches, per-branch settings, staff controls, activity log, Zypher support consent

Zypher console

- Overview (fees, collections, overdue, unused shops); shop list and shop page (plan, fee, notes, pause, business types, feature switches, people, payments)
- New-shop 4-step wizard; billing (receipts, WhatsApp reminders); business types; audit log; Zypher team; settings (company/bank, auto-pause)
- Plans: Starter Rs 499, Pro Rs 999, Business Rs 1,999 per month before GST (differ by branches, logins, pieces); 14-day trial

Decisions made (Launch Pack 1, 11 Nov 2026)

- Late fee: fixed Rs per piece per day OR % of agreed daily price; per-product override; optional cap; rules copied onto each booking (late_terms)
- Late piece vs next booking: warn, offer swap, or hold the next hand over with a note
- Invoice timing chosen per shop; printing never assigns a number; bank details concealed (last 4 only)
- Deleting a booking is 'cancel with a reason', not a real delete

11. Testing, CI and operations

Area	Details
pgTAP (supabase/tests)	22 files: engine, security, products, operations, platform, owner_staff, zypher_setup_only, launch, activity, late_returns, branch_settings, staff_controls, invoice(s)_receipts, cleaning, service_jobs, sku, support_visits, expenses, features, attributes, reactivate. Last recorded: 429 passing (10 Nov); plan sum 575 now.
Playwright (tests/)	smoke, v2-shop, bridal-flow, launch, mfa (30 tests). Need Docker local Supabase + fixtures; not run since the move to shared cloud project.
CI (ci.yml)	app job: tsc --noEmit, lint, build with placeholder keys; database job: local Supabase, db reset, fixtures, supabase test db; browser job after both.
Backups	backup.yml: daily encrypted dump as private artifact for 30 days (switched on by BACKUP_ENABLED); plus Supabase's own 7 days. Restore steps in docs/RUNBOOKS.md.
Dependabot	Automatic dependency PRs (many merged in git log)
Environments	Shared cloud Supabase for dev (3 developers). Docker only for throwaway tests. Nothing deployed yet: no staging/production, no email sender.

Workflow rules (CLAUDE.md)

- Never connect to TRENDZ projects; never run db reset / seed / fixtures / bulk inserts on the shared project
- Every DB change is a migration committed to git; ask how to apply (SQL Editor, 'db push', or Claude via connected account)
- Plan first, wait for approval; no new dependencies without asking; never read or print .env files
- Feature branches + PRs (feature/invoices, feature/cleaning-repairs...), merged into main

12. Likely interview questions - short answers

Q: Why no backend server?
A: Rules live in Postgres functions + RLS. One source of truth for web, mobile and future clients. Fewer moving parts and a security boundary that the browser cannot bypass.

Q: How do you isolate tenants?
A: shop_id on every row, restrictive RLS tied to current_shop_id(), composite FKs (shop_id, x_id), and shop functions run as a NOBYPASSRLS role (app_rpc).

Q: How do you stop double booking?
A: A btree_gist exclusion constraint on unit_reservations time ranges (rental period plus turnaround/cleaning). The DB rejects overlaps even under concurrency.

Q: How do you keep money correct?
A: numeric(14,2); CHECK constraints for total, paid, deposit; triggers derive paid from payments; payments are voided not deleted; reports sum the records.

Q: What does the browser trust?
A: Nothing. Server actions call get_my_role() from the DB; DB functions re-check role, shop and branch; validation runs on both sides.

Q: How does the module system work?
A: business_templates rows (data) + src/modules/ + registry.ts. ESLint forbids cross-module imports; one-line route files keep URLs flat.

Q: How did you do auditing?
A: Database triggers write activity_log in plain sentences; browser cannot write it; log function swallows errors so it never blocks a rental; separate audit_log for setup and corrections.

Q: How is auth handled?
A: Supabase Auth with SSR cookies; proxy.ts verifies JWT via getClaims(); invite links by service-key server actions; TOTP MFA (aal2) required for Zypher staff and support.

Q: How did you handle late fees?
A: Owner settings (mode, grace cut-off, 24h blocks, cap, per-product override) copied onto each booking as late_terms so later setting changes do not alter old bookings; return screen shows the working.

Q: What would you improve?
A: Move Finance aggregation to a DB function for big shops; typed queries; hosting + staging; monitoring; backups beyond 7 days; partial returns; data export/offboarding; vehicle and stay modules; mobile app.

Q: What was hard?
A: Keeping rules in SQL while keeping the UI friendly; migrations on a shared DB; keeping the activity log complete for every new feature; clothing-specific flows (piece-level booking, cleaning days) without hurting other business types.

Q: Tests?
A: pgTAP for rules and security (cross-shop isolation, roles, money), Playwright for flows (booking to return, MFA), CI runs typecheck, lint, build, DB and browser tests.

Open / not done (be honest if asked)

- Not deployed; no staging or production; no email sender; no error monitoring
- Partial returns, shop data export, voiding a wrong Zypher payment, usage page, announcements, offboarding not built
- Car, bike and resort screens are the older generic ones, less tested; vehicle and stay modules and mobile app are future (docs/APP_CONTRACT.md)
- Accountant sign-off pending on GST wording (forfeited advance, time of supply, supplementary bills, SAC code)
- Database and Playwright tests last run before the move to the shared project; rerun on Docker